LOCH AND LOADED: DIGITAL

Technical Design Document

Engine: Godot 4.5.1 | Language: GDScript | Platform: Windows

Source Control: GitHub

Table of Contents

Table of Contents 2
1. Project Overview 4
  1a. Game Concept 4
  1b. Technology Stack 4
  1c. Technical Risks 4
2. Game Play 5
  2a. Game Structures and Objects 5
  2b. Physics 5
    Movement Constants 5
    Gravity 6
    Jump System 6
    Wall Interaction 6
    Dash 6
    Slide and Crouch 6
    Ground Slam 6
    Zipline 7
    Crane / Yoyo Traversal 7
    Collision Layers 7
  2c. Projectiles 7
  2d. Collision 7
  2e. Player Actions 7
  2f. Victory Conditions 8
3. Game Systems 9
  3a. Spray Spot 9
  3b. Custom Spray Editor 9
  3c. Scene Transitions 9
  3d. Healing 9
  3e. Spray Resource 9
  3f. Stun Attack 9
  3g. Slam Attack 10
  3h. Yoyo Attack 10
  3i. Player System 10
  3j. Enemy System 10
  3k. Combat System 11
  3l. Input System 11
4. Artificial Intelligence 12
  Shared Behaviour: Patrol and Detection 12

4a. Basic Robot 12
Patrol 12
Attack — Zap 12
4b. Flybot 13
Patrol 13
Attack — Bullet Fire 13
Stun Interaction 13
4c. Charger 13
Patrol 13
Attack — Three-Bounce Cycle 13
5. Code Overview 15
5a. Main Game Loop 15
5b. Core Systems 15
5c. Data Flow 15
5d. Node Architecture 15
6. User Interface 17
6a. Game Menus 17
Startup Menu 17
Spray Editor 17
Pause Menu 17
Settings Menu 17
6b. Player HUD 17
7. Audio 18
8. Shaders 19
9. Performance 20
9a. Targets 20
9b. Rendering 21
9c. VRAM Management 22
10. Source Control 23
10a. Branching Strategy 23
10b. Commit Conventions (Retrospective) 23
10c. .gitignore Configuration 23
11. Dialogue System 24
12. Known Issues 25
13. Future Improvements 26

1. Project Overview

1a. Game Concept

Loch and Loaded: Digital (working title: Radical Rampage) is a 2D action platformer built in Godot 4. The game centres on Tuff Tiff, a player character who traverses urban rooftops, fights robotic enemies and leaves personal graffiti tags across the environment. The design focus is on responsive, layered movement combined with a varied combat toolkit and traversal mechanics.

1b. Technology Stack

Component	Details
Engine	Godot 4.5.1
Primary Language	GDScript
Shader Language	GDShader
Source Control	GitHub
Target Platform	Windows (primary), Web (secondary)
Addons	None

1c. Technical Risks

Risk	Mitigation
High-resolution texture loading (16k) stalls on Web builds	Cap textures at 4k where possible; 8k absolute maximum. Break large images into tiles and stream per-frame to avoid single-frame spikes.
VRAM budget exceeded on mid-range GPUs (8 GB ceiling)	Import textures with Godot compression. Avoid duplicating large assets. Spread heavy assets across scene transitions.
Web build incompatibility with 16k textures	All textures targeting web must be 4k or below. Test web build separately in CI.
Physics body swap on enemy death causing stutter	Pre-instance ragdoll bodies; swap via deferred call to avoid mid-frame collision recalculation.
Spray editor canvas performance with large brushes	Cache brush shape as a pre-rasterised bitmap; apply alpha blend in a single draw pass per stroke.

2. Game Play

2a. Game Structures and Objects

The game world is a side-scrolling urban environment composed of the following structural categories:

- Buildings (varied height): Serve as the primary traversal surface. The player can run across rooftops, jump between windows, and wall-jump on vertical faces.
- Speakers: Ambient audio emitters placed throughout the level. Volume scales with proximity to the player using Godot's AudioStreamPlayer2D distance attenuation.
- Spray Spots: Fixed interactable nodes that accept the player's custom spray texture. Each has a proximity trigger (Area2D) and stores whether it has been tagged.
- Ziplines: Two-node traversal objects. The player enters a detection area to grip; velocity is locked to the line direction at a fixed speed.
- Cranes: Single-point traversal targets reachable via the yoyo. On yoyo attachment the player is pulled toward the crane anchor point.
- Play Area Boundary: A single RectangleShape2D Area2D that both constrains camera limits and triggers respawn when the player exits its bounds.
- Level End Area: An Area2D placed at the goal; entering it with the Player body triggers the end-of-level screen.

Character roster:

- Tuff Tiff: The player character (CharacterBody2D).
- Basic Robot: Ground-roaming enemy.
- Flybot: Airborne enemy that fires projectiles at the player.
- Charger: Charges up an attack before launching at the player.

2b. Physics

The player character uses Godot's built-in CharacterBody2D move_and_slide() system. The following constants and behaviours are derived directly from the Player script:

Movement Constants

Constant	Value / Description
MAX_SPEED	180 px/s — horizontal speed cap (halved when crouching)
TIME_TO_MAX	0.4 s — time to reach max speed from rest
ACCEL	MAX_SPEED / TIME_TO_MAX = 450 px/s^2 — applied per frame via move_toward()
FRICTION	600 px/s^2 — deceleration when no directional input
JUMP_VELOCITY	-400 px/s — initial upward velocity on jump
JUMP_CUT_MULTIPLIER	0.4 — velocity multiplied when jump is released early (variable height)
SLAM_SPEED	900 px/s — downward velocity during ground slam
SLIDE_SPEED	280 px/s — minimum horizontal speed on slide start
SLIDE_FRICTION	180 px/s^2 — deceleration during a slide
SLIDE_MIN_SPEED	30 px/s — slide ends when speed drops below this

CROUCH_SPEED_MULT	0.5 — max speed multiplier while crouching

Gravity

Gravity is read from Godot's project setting physics/2d/default_gravity and cached at _ready(). A gravity multiplier of 1.5 is applied when the player is falling (velocity.y > 0) to produce a snappier arc. During a ground slam the Y velocity is overridden to SLAM_SPEED and horizontal velocity is zeroed, bypassing gravity.

Jump System

The jump system layers three sub-systems:

- Coyote Time (0.22 s): After leaving a floor or wall, the player retains the ability to jump for COYOTE_TIME seconds. A timer counts down from the last grounded frame; if it expires before a jump is pressed, one jump count is consumed.
- Jump Buffering (0.1 s): Pressing jump stores the intent for JUMP_BUFFER_TIME seconds. If the player lands within that window, the jump executes immediately. This is implemented via a countdown timer decremented in _handle_jump() each physics frame.
- Variable Jump Height: Releasing the jump button while still ascending multiplies velocity.y by JUMP_CUT_MULTIPLIER (0.4). This is handled in _input() on the action_released event, not in the physics loop.
- Double Jump: MAX_JUMPS = 2. jumps_left is reset to 2 on landing or wall contact and decremented each jump.

Wall Interaction

Two RayCast2D nodes (top_ray and bottom_ray) detect wall contact independently. When both are colliding and the player is airborne with downward velocity, gravity is damped by a factor of 3.2 (if the player is pushing into the wall) or 1.2 (otherwise), producing a wall-slide effect. While wall-sliding, jumps_left is reset to MAX_JUMPS, enabling a wall-jump. The wall jump blends the wall normal direction (scaled to 400 px/s) with the current velocity.

Dash

A dash locks the player into a directional burst for DASH_DURATION (0.2 s) at 400 px/s. The cooldown is DASH_COOLDOWN (1.0 s). Direction is horizontal based on facing direction; if airborne and stationary, the dash fires upward. Gravity and jump input are suppressed during the dash. A dash attack area becomes active for the duration and attaches the yoyo to any enemy hit.

Slide and Crouch

Crouching and sliding share the crouch input. If horizontal speed exceeds 60 px/s at crouch input, a slide begins. The idle collision shape is disabled during both states. Sliding applies SLIDE_FRICTION until speed drops below SLIDE_MIN_SPEED or the player leaves the floor. Crouch re-enables the collider only if a headroom raycast (32 px upward) finds no obstruction.

Ground Slam

Initiated when airborne and not already slamming or dashing. Sets velocity to Vector2(0, SLAM_SPEED). On landing, slam_attack_area becomes active for 0.1 s, then SLAM_END_LAG (0.15 s) of end lag is applied via await before control is restored. The slam has a SLAM_COOLDOWN of 3.0 s. Screen shake is applied on impact if the screen shake setting is enabled.

Zipline

The player enters a zipline via an Area2D detector. The zipline parent node exposes a get_dir() method returning the normalised line direction. While on a zipline, gravity is bypassed and velocity is locked to zipline_dir * 400 px/s. If the player moves downward on the analog stick (dir_radial.y >= 1.0), the grip is released.

Crane / Yoyo Traversal

The crane exposes a get_point() anchor position. When the yoyo is thrown at a crane anchor (detected via _in_crane_area), _on_crane is set and the player is pulled toward the point using crane_dir * 5 px/s per frame. On reaching within 50 px, _on_crane is cleared and velocity receives a final impulse in the approach direction.

Collision Layers

The player's collision mask is manipulated at runtime:

- Layer 2 is disabled while the player is holding downward input (to drop through one-way platforms) and re-enabled when released.
- Layer 2 is also disabled during crouch when a downward joystick angle is detected (89-91 degrees), allowing drop-through from a crouched position.

2c. Projectiles

Flybot enemies fire projectiles at the player. Projectiles are standalone CharacterBody2D instances They travel at a fixed velocity toward the player's position at fire time. On collision with the player they call a damage method and queue_free(). Lifetime is capped via a Timer node to prevent orphaned projectiles if they miss.

2d. Collision

The following collision architecture is used:

- Player vs. Environment: CharacterBody2D with move_and_slide(). Two ray casts supplement the built-in wall detection for wall-slide logic.
- Player vs. Enemy Bodies: Area2D attack areas on the player (attack_area, dash_attack_area, stun_area, slam_area) detect enemy CharacterBody2D children via body_entered signals.
- Enemy vs. Player: Enemies detect the player using their own Area2D or direct CharacterBody2D contact depending on type.
- One-Way Platforms: Handled via collision layer masking (layer 2) toggled at runtime.
- Play Boundary: An Area2D wrapping the level detects when body == self (player) exits, triggering respawn.

2e. Player Actions

Action	Behaviour
Move Left / Right	Accelerates to MAX_SPEED via ACCEL; FRICTION applied when no input
Jump	Buffered, coyote-enabled, variable height, double-jump supported

Dash	1 s cooldown; 0.2 s duration at 400 px/s; activates dash attack area
Crouch / Slide	Slide if speed > 60 px/s, else crouch; disables idle collider
Ground Slam	Requires airborne; 900 px/s down; 3 s cooldown; AoE on land
Punch	Instant AoE in front of player; 0.4 s active window
Stun Attack	AoE pulse; stuns enemies for 3.3 s; 5 s cooldown
Ready Yoyo	Hold to display aim ring; releasing without throw hides it
Throw Yoyo	Fires yoyo toward aim dir; attaches to first enemy or crane hit
Fire Yoyo (attached)	Damages attached enemy (100 dmg) and pulls them toward player
Spray	Plays spray animation; triggers spray spot painting
Drop Through Platform	Hold down input to disable layer 2 collision

2f. Victory Conditions

A level is considered complete when the player's CharacterBody2D enters the level_end_area (Area2D). The level end screen widget is made visible and the parent level node's endOfLevel() method is called to handle any scoring, persistence, and scene transition logic.

3. Game Systems

3a. Spray Spot

A fixed world node the player can interact with when in range (detected via Area2D proximity). Pressing the assigned interaction key with sufficient spray resource triggers a GPUParticles2D animation simulating spray paint forming. Once the animation completes, the particles are removed and a Sprite2D takes their place using the player's custom (or default) spray texture. The spot records its tagged state to prevent re-tagging.

3b. Custom Spray Editor

A standalone in-engine canvas (accessible from the main menu) that allows the player to paint a 512x512 image. Features:

- Multiple stylus shapes (brush types).
- Colour picker and adjustable brush size.
- Alpha blending on paint: colours mix with existing paint beneath the brush.
- Eraser mode: overrides existing colour rather than blending.
- Brush shape is cached on modification to avoid recalculating geometry each stroke.
- Output is saved as a PNG (saved.png) in Saves/Spray/ and loaded as a texture at runtime by the Spray Spot system.

3c. Scene Transitions

A global autoloaded Node (SceneTransition) sits as the root scene and manages all transitions. It plays a transition animation using Signed Distance Fields to generate a stylised wipe graphic. The same node exposes a death_reset() method called by the player on death, which reloads the current scene after playing the animation.

3d. Healing

The player's _heal_player() function is called on enemy kill. It restores 35% of missing health plus a flat 10 HP, clamped to MAX_HEALTH. This design slows damage accumulation without granting full recovery, rewarding aggressive play.

```
health_value += (missing_health * 0.35) + 10.0
health_value = clamp(health_value, 1, MAX_HEALTH)
```

3e. Spray Resource

A per-player resource counter gating spray spot usage. It is incremented via _addSpray() (which emits the spray_changed signal consumed by the UI). Resource is earned by defeating enemies. The UI displays current spray count and the interaction prompt is hidden if the player has insufficient resource.

3f. Stun Attack

An AoE pulse centred on the player. On activation, stun_attack_area (Area2D) is set to monitoring for 1 second. Any enemy body that enters during this window has its stun_timer set to 3.3 s, preventing movement and disabling flight for Flybots. A shader is applied to the player AnimatedSprite2D to produce an EMP visual. Cooldown is 5 s.

3g. Slam Attack

A ground AoE triggered on landing from a ground slam. slam_attack_area becomes active for 0.1 s on floor contact. Enemies in range receive 30 damage, an upward knockback impulse, and a 1.65 s stun (half of STUN_TIME). The player is locked in end lag (0.15 s) before regaining control.

3h. Yoyo Attack

A multi-purpose targeted weapon system with three modes:

- Dash Through Enemy: When the dash attack area contacts an enemy body, the yoyo attaches to that enemy and is visible at the enemy's position. Pressing Fire Yoyo deals 100 damage and pulls the enemy toward the player.
- Thrown Yoyo: If yoyo_ready is active without an enemy in the detector, the yoyo is thrown toward the aim direction (right stick on controller, mouse cursor on keyboard). It travels to a fixed distance (150 px from player) and returns automatically after _yoyo_duration (1.0 s).
- Crane Attachment: If _in_crane_area is true and Fire Yoyo is pressed, the yoyo attaches to the crane anchor and initiates crane traversal.

The yoyo position is drawn using a Sprite2D; a Line2D (yoyo_string) connects the player origin to the yoyo each frame. The aim ring (Sprite2D) is rotated to match the aim direction each process frame when ready.

3i. Player System

Core CharacterBody2D handling movement, combat, and traversal. Key implementation details are covered under Section 2b (Physics) and 2e (Player Actions). Additional notes:

- Easy Mode: If Global.getGameModeEasy() returns true at _ready(), MAX_HEALTH is doubled before health_value is set.
- Camera Limits: Automatically computed from the bounding RectangleShape2D of play_area_shape in _setup_camera_limits(). Camera smoothing is enabled.
- Screen Shake: A random offset is applied to Camera2D each frame. shake_strength decays exponentially (lerp toward 0 at shake_decay_rate). apply_shake() resets it to random_shake_strength.
- Signals: health_changed(float), spray_changed(float), and cooldowns(float, float, float) are emitted each frame for the HUD.

3j. Enemy System

All enemies extend a shared base class (Enemy). The base class provides:

- Health and damage handling via take_hit() and take_dash().
- Immunity frames to prevent hit-stacking.
- stun_timer: a countdown that locks movement and flight while > 0.
- Death transition: on health reaching zero the enemy CharacterBody2D is replaced by a RigidBody2D counterpart (pre-instanced as a sibling node) to produce a ragdoll that the player can physically interact with.
- Reward system: death calls the player's _heal_player() and increments the score.

Enemy subtypes (Basic Robot, Flybot, Charger) each have their own alive CharacterBody2D sprite/physics nodes and a corresponding dead RigidBody2D ragdoll. The node architecture keeps both representations in the scene tree simultaneously; only one is active at a time.

3k. Combat System

Attack	Damage / Effect
Punch	150 damage, directional knockback + upward bias
Dash Attack	40 damage, upward knockback; attaches yoyo to target
Slam Attack	30 damage, upward knockback, 1.65 s stun
Stun Attack	No damage; 3.3 s stun, disables flight
Yoyo Fire	100 damage, pulls enemy toward player

3l. Input System

The input system supports keyboard/mouse and controller simultaneously with runtime remapping and persistent storage.

- Controller Detection: The controller name string is queried via Input.get_joy_name(0).to_lower() and checked against known substrings (xbox, microsoft, playstation, dualshock, dualsense, ps4, ps5, switch, nintendo, pro controller). This determines whether the right stick or mouse cursor is used for aiming.
- Runtime Remapping: Actions can be rebound in the Controls tab of the pause menu. Changes apply immediately to Godot's InputMap.
- Persistence: Remapped controls are saved using Godot's ConfigFile API and reloaded on startup.
- Analog Crouch-Drop: Joystick angle is checked (89-91 degrees) to distinguish a pure downward input for platform drop-through from a diagonal that should only trigger crouch.

4. Artificial Intelligence

All three enemy types share the same two-phase behavioural structure: a passive patrol phase and an active attack phase. The transition between phases is driven by a single proximity check against the player. No pathfinding is used; movement is purely horizontal or physics-driven. A formal state machine is planned as a future improvement to replace the current per-type boolean flag implementation.

Shared Behaviour: Patrol and Detection

While the player is outside the enemy's detection radius, every enemy type executes its patrol behaviour. Detection is implemented as a circular Area2D (or distance comparison) centred on the enemy. Each physics frame the enemy checks whether the player's position falls within this radius. Once the player enters range the enemy transitions to its attack phase and does not return to patrol until the player leaves range or the enemy completes a cooldown cycle.

Phase	Trigger Condition
Patrol	Player outside detection radius (default state)
Attack	Player enters detection radius
Cooldown (Charger only)	After completing three bounce cycles

4a. Basic Robot

Patrol

The Basic Robot moves horizontally between two patrol boundary points defined in the editor. When it reaches either boundary it reverses direction. The robot stays grounded and does not attempt to navigate drops or platforms; it simply turns at the edge of its patrol range currently said boundary points are walls or edges of platforms.

Attack — Zap

When the player enters the robot's detection radius the robot stops moving and executes a zap attack. The zap is a short-range electrical discharge directed at the player. It is implemented as a hitbox (Area2D) that activates for a brief window each attack cycle, dealing damage on contact. After the attack resolves there is a short recovery pause before the robot checks detection again. If the player is still in range the cycle repeats; if the player has left range the robot resumes patrol.

Robot — Key Properties	Description
Patrol	Moves between two boundary markers; reverses on reaching either end
Detection	Proximity check each physics frame; single-radius trigger
Attack Type	Zap — short-range electrical hitbox, area-based activation window
Post-Attack	Brief recovery before re-checking detection; returns to patrol if player has left
Stun Interaction	Stun timer > 0 freezes movement and prevents attack; robot falls if airborne

4b. Flybot

Patrol

The Flybot hovers at a fixed vertical offset above the ground, drifting horizontally within its patrol range in the same boundary-reversal pattern as the Basic Robot. Its vertical position is held constant during patrol; no gravity applies while it is in flight.

Attack — Bullet Fire

When the player enters detection range the Flybot locks its horizontal drift and begins firing projectiles. Each projectile is spawned at the Flybot's position and travels in a straight line toward the player's position at the moment of firing (no tracking after spawn). The Flybot fires at a fixed interval on a repeating timer. It continues firing for as long as the player remains in range.

Stun Interaction

When the Flybot's stun_timer is set (e.g. by the player's stun attack) its flight behaviour is suspended and gravity is applied, causing it to fall to the ground. While grounded and stunned it cannot fire. This creates a deliberate window for the player to deal damage using ground-based attacks such as the slam or punch, which would not normally reach an airborne target.

Flybot — Key Properties	Description
Patrol	Horizontal drift at fixed height; reverses at boundary markers
Detection	Proximity check; transitions from drift to stationary firing stance
Attack Type	Projectile — straight-line bullet toward player position at fire time
Fire Rate	Fixed interval timer; continuous while player in range
Stun Interaction	Gravity enabled, flight suspended, falls to ground; cannot fire while stunned

4c. Charger

Patrol

The Charger is a wheeled enemy that remains stationary during its patrol phase. It does not drift or move between boundaries. It sits idle, waiting for the player to enter its detection radius.

Attack — Three-Bounce Cycle

When the player enters range the Charger launches into its attack sequence. The attack consists of three consecutive bounces:

- Bounce 1: The Charger launches toward the player's current position at high speed. On contact with a surface (wall, floor, or ceiling) it rebounds using physics reflection.
- Bounce 2: After the first rebound the Charger continues moving and bounces a second time off the next surface it contacts.
- Bounce 3: The Charger bounces a final third time before its velocity decays to zero and it comes to rest.

If the Charger contacts the player at any point during the three bounces it deals damage and applies knockback. After the third bounce has resolved the Charger enters a cooldown state during which it is

stationary and cannot attack. Once the cooldown expires it returns to its idle patrol state and will re-trigger if the player is still in range.

Charger — Key Properties	Description
Patrol	Stationary idle; no horizontal movement during patrol phase
Detection	Proximity check triggers the three-bounce attack cycle
Attack Type	Physics-driven bounce — reflects off surfaces up to three times
Player Contact	Deals damage and knockback on any bounce that contacts the player
Post-Attack	Cooldown period (stationary, cannot attack); then returns to idle detection loop
Stun Interaction	Stun timer interrupts the bounce cycle mid-sequence; velocity zeroed

Note: A formal state machine is planned as a future improvement to replace the current per-type boolean flag approach used across all three enemy types. This would make adding new enemy types or modifying existing transitions significantly easier.

5. Code Overview

5a. Main Game Loop

Godot's built-in loop is used without modification. The Player script separates concerns across three callbacks:

- _input(event): Handles discrete input events (jump buffer, dash start, attack triggers, yoyo ready/throw).
- _process(delta): Non-physics per-frame updates: screen shake, damage shader, yoyo position, cooldown signal emission, health check.
- _physics_process(delta): Deterministic movement: gravity, jump, horizontal movement, wall rays, animation state, move_and_slide().

5b. Core Systems

System	Responsibility
Player (player.gd)	Movement, combat, traversal, signals to HUD
Enemy (enemy.gd)	Base class: health, stun, death, ragdoll swap, rewards
SceneTransition (autoload)	SDF wipe animations, scene reload, death reset
Global (autoload)	Game mode flags (easy mode), persistent settings
Input System	Runtime remapping, ConfigFile persistence, controller detection
Spray Editor	Canvas painting, brush caching, alpha blend, PNG export
Dialogue System	JSON-driven NPC lines, speaker field, cooldown between talks

5c. Data Flow

The following describes the primary data paths through the game:

- Player state -> HUD: Signals (health_changed, spray_changed, cooldowns) emitted from Player each frame; HUD nodes connect to these and update UI elements.
- Enemy death -> Player heal + Score: Enemy base class calls player._heal_player() and increments the scene-level score counter on death.
- Spray Editor -> Spray Spot: The editor writes saved.png to disk. At runtime, spray spots load this file as a texture via load() or ResourceLoader.
- Input remapping -> Persistence: Changes written to a ConfigFile at Saves/save_data.txt; read back on startup via Global.
- Dialogue JSON -> NPC: The NPC reads a JSON file line-by-line using the event string as a key and displays speaker + text. After all lines play, a cooldown timer prevents re-triggering.

5d. Node Architecture

Player node hierarchy:

- CharacterBody2D (Player)
  - Camera2D — with computed limit bounds
  - AnimatedSprite2D — ShaderMaterial for damage and stun visuals

    - AnimationPlayer (effect_player) — slam, stun animations
    - CollisionShape2D (idle_collider) — disabled during crouch/slide
    - Area2D nodes: attack_area, dash_attack_area, stun_area, slam_area, zipline_detector, crane_detector, yoyo_detector
    - RayCast2D nodes: top_ray, bottom_ray (wall detection)
    - Sprite2D: yoyo, ring; Line2D: yoyo_string
    - AudioStreamPlayer2D: one per sound effect type

Enemy node hierarchy (per enemy type):

- Node2D (root)
    - CharacterBody2D (_alive) — active during live state; contains sprite, collision, AI logic
    - RigidBody2D (_dead) — active on death; pre-configured as ragdoll
    - CollisionShape2D — shared shape definition (one per physics body)

6. User Interface

6a. Game Menus

Startup Menu

Five buttons presented on a 2D canvas scene:

- Play: Loads the game level via SceneTransition.
- Spray Editor: Opens the spray editor interface.
- Settings: Opens the settings menu.
- Credits: Displays the credits screen.
- Exit: Calls get_tree().quit().

Spray Editor

Full-screen canvas with the following controls: stylus shape selector, colour picker, brush size slider, eraser toggle. The brush is cached on change. Output is saved to Saves/Spray/saved.png on confirm and excluded from version control via .gitignore.

Pause Menu

A popup Control node. On open, the scene tree is paused (get_tree().paused = true). Contains three tabs:

- Sound: Master, Music, and SFX volume sliders. Music and SFX feed into the master bus after their own attenuation.
- Accessibility: Toggles for screen shake, film grain; regular vs. accessible mode; four colour blind presets (None, Deuteranopia, Tritanopia, Protanopia) applied via a GDShader post-process.
- Controls: Displays the full input map for keyboard/mouse and controller. Clicking a binding opens a remap prompt. A reset-to-defaults button is provided.

Settings Menu

Accessible from the startup menu. Contains the same Sound and Accessibility tabs as the pause menu. Changes are written to ConfigFile immediately. No Controls tab in this context (remapping is done in-game).

6b. Player HUD

The HUD is a CanvasLayer that connects to the Player node's signals on scene load:

- Health Bar: Driven by health_changed signal. Displays current HP as a progress bar; colour shifts at low health thresholds.
- Spray Counter: Driven by spray_changed signal. Displays available spray as a fraction with a spray icon
- Cooldown Indicators: Driven by cooldowns(dash, slam, shock) signal emitted each frame. Three radial or bar indicators show remaining cooldown as a 0-1 ratio.

7. Audio

All audio uses Godot's AudioStreamPlayer2D for positional sound (proximity-based attenuation) and AudioStreamPlayer for non-positional audio (UI, background ambience). The audio bus structure is:

- Master Bus: Final output; controlled by the master volume slider.
- Music Bus: Background tracks routed through master; controlled by the music slider.
- SFX Bus: All in-game sound effects routed through master; controlled by the SFX slider.

The following sound effects are attached to the Player node:

Node	Trigger
footstep_sfx	Random chance (~20%) each run animation frame when on floor
dash_sfx	Dash start
punch_sfx	Punch input
YoYo_sfx / yoyoThrow_sfx	Yoyo fire and throw events
stunAttack_sfx / stunRecharge_sfx	Stun start; stun cooldown complete (1 s delay)
jump_sfx	Every jump
slam_sfx	Landing from a ground slam
backgroundAmb	Plays on _ready(); loops continuously

8. Shaders

GDShader is used for the following effects:

Shader	Description
Player damage shader	ShaderMaterial on AnimatedSprite2D. taking_damage parameter flashes the sprite on hit. stunning parameter applies an EMP pulse visual during stun.
Dash particles shader	Applied to GPUParticles2D. fliph parameter mirrors particle direction to match facing.
Scene transition (SDF wipe)	Full-screen post-process on the SceneTransition CanvasLayer. Uses a signed distance field shape to animate a stylised lava lamp effect between scenes
Colour blind mode shader	Post-process applied to the root viewport. Swaps colour channels per-pixel based on the selected mode (deuteranopia, tritanopia, protanopia).
Film grain (optional)	Full-screen half-tone overlay; toggled in accessibility settings.

9. Performance

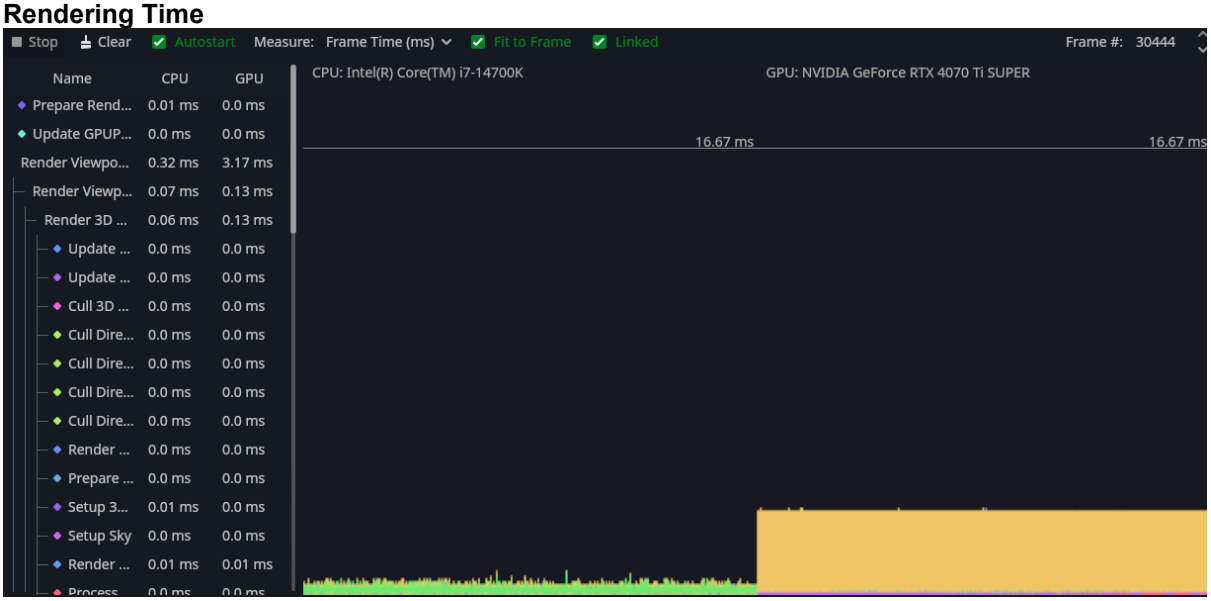
9a. Targets

Metric	Target
Frame Rate	Stable 60 FPS on target hardware
1% Low (observed)	120 FPS (8 ms worst frame) in debug sampling
VRAM Budget	< 8 GB (ceiling of mid-range consumer GPUs such as RTX 3060 / RX 6600)
Max Texture Size (Web)	4k (8k absolute maximum)
Max Texture Size (Desktop)	8k (4k preferred)

9b. Figures

Frame Time

Rendering Time



VRAM

9c. Rendering

The game uses Godot's 2D renderer exclusively. Contributing factors to low rendering overhead:

- Area2D used for combat detection rather than physics bodies (avoids collision response overhead).
- Physics body swap on enemy death via deferred call prevents mid-frame collision recalculation.
- Brush bitmaps cached in the spray editor to avoid per-stroke geometry recalculation.
- Texture sizes capped to reduce GPU upload time on scene transitions.

9d. VRAM Management

Exceeding 8 GB VRAM on target GPUs causes texture paging to system RAM, resulting in visible stutters and texture pop-in. The project originally used 16k textures; these were incompatible with the web build and caused memory pressure on desktop. Textures are now capped at 4k where visual difference is negligible.

A single uncompressed 8k RGBA texture occupies approximately 256 MB of VRAM; a 4k equivalent uses roughly 64 MB — a fourfold reduction. For a 2D game at typical screen resolutions, 4k textures are visually sufficient.

Additional VRAM controls:

- Godot compression settings applied at import rather than storing raw uncompressed data.
- No duplicate texture assets across sprites.
- Heavy assets distributed across scene transitions to avoid simultaneous loading.

10. Source Control

10a. Branching Strategy

All development was committed directly to a single main branch. Given the small team size, low frequency of conflicting commits, and the availability of team members to resolve conflicts in real time, the overhead of a formal branching strategy was judged to outweigh its benefits for this project.

10b. Commit Conventions (Retrospective)

Commits were made without a standardised format. For future projects the team would adopt a prefix convention:

Prefix	Usage
[Feature]	New functionality added
[Fix]	Bug or error resolved
[Asset]	Art, audio, or other non-code files added or updated
[Refactor]	Code restructured without behaviour change
[Docs]	Documentation updates

10c. .gitignore Configuration

The project was started in Visual Studio, requiring the default Godot .gitignore to be extended with Visual Studio exclusions. Save data (custom spray and control settings) are explicitly excluded to prevent runtime user files from entering the repository.

```
# Godot 4+
.godot/  |  .import/  |  .nomedia
# Runtime save data (user-generated, excluded from repo)
Saves/Spray/saved.png
Saves/save_data.txt
# Visual Studio
.vs/  |  *.user  |  *.suo  |  *.sln.docstates
```

11. Dialogue System

The dialogue system currently supports a single NPC but is built for extensibility. Data is stored in a JSON file where each entry uses an event string as the key and contains the speaker name and dialogue line. The NPC node reads and displays lines sequentially. After all lines in a sequence have played, the NPC enters a cooldown period before it can be triggered again. The UI displays the speaker name and dialogue text using a standard Control node, animated in and out per line.

Planned extensions: multiple concurrent NPCs, branching dialogue choices, integration with the score/progression system to gate certain dialogue lines.

12. Known Issues

Issue	Detail / Workaround
16k textures fail on Web builds	Textures must be broken into tiles at 4k or below for web compatibility. Desktop builds tolerate 8k with memory pressure.
Formal state machine absent	Enemy AI uses Boolean flags. A state machine refactor is planned.
Single NPC dialogue	Dialogue system is implemented for one NPC. Multi-NPC support requires extending the JSON schema and NPC base class.
No input abstraction layer	Input actions are queried directly by name in player code. A formal input manager would improve testability.

13. Future Improvements

- Formal enemy state machine to replace current boolean flag approach.
- Expanded AI behaviours (patrol paths, group coordination, difficulty scaling).
- Improved input abstraction layer with a dedicated InputManager singleton.
- Multi-NPC dialogue with branching and progression gating.
- Streaming texture loader to eliminate per-frame load spikes on high-resolution assets.
- Controller haptics via Input.start_joy_vibration() on hit events.
- Additional traversal mechanics (grapple hooks, bounce pads).
- Formal scene state machine to manage startup, gameplay, pause, and death states.

— End of Document —